

24BCF10059 - Lakshdeep Experiment 1

Aim :

To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, subqueries, self-joins, and date operations to solve real-world database problems.

Question 1

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

Solution :

```
SELECT D.dept_name AS "Department" , ROUND(AVG(marks_scored),2) AS "Average Score"
```

```
FROM Departments AS D
```

```
INNER JOIN
```

```
Students AS S
```

```
ON D.dept_id = S.dept_id
```

```
INNER JOIN
```

```
Marks AS M
```

```
ON S.student_id = M.student_id
```

```
GROUP BY D.dept_id , D.dept_name
```

```
ORDER BY "Average Score" ASC;
```

Question 2

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

Table A

EmpID	Ename	Salary
1	AA	1000
2	BB	300

Table B

EmpID	Ename	Salary
2	BB	400
3	CC	100

Solution :

```
SELECT EmpID,  
       Ename,  
       MIN(Salary) AS Salary  
FROM (  
    SELECT * FROM A  
    UNION ALL  
    SELECT * FROM B  
) AS INTERMEDIATE_RESULT  
GROUP BY EmpID, Ename  
ORDER BY Ename;
```

Question 3

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

Table A

EmpID	Ename	Salary
1	AA	1000
2	BB	300

Table B

EmpID	Ename	Salary
2	BB	400
3	CC	100

Solution :

```
SELECT E.name AS E_NAME , D.dept_name AS DEPARTMENT , E.salary AS SALARY
FROM Employee AS E
INNER JOIN
Department AS D
ON E.department_id = D.id
WHERE E.Salary IN(
    SELECT MAX(E1.salary)
    FROM Employee AS E1
    WHERE E1.department_id = E.department_id
);
```

```

SELECT E.name AS E_NAME,
       D.dept_name AS DEPARTMENT,
       E.salary AS SALARY
FROM Employee AS E
JOIN Department AS D
ON E.department_id = D.id
WHERE E.salary = (
    SELECT MAX(E1.salary)
    FROM Employee AS E1
    WHERE E1.department_id = E.department_id
    AND E1.salary < (
        SELECT MAX(E2.salary)
        FROM Employee AS E2
        WHERE E2.department_id = E1.department_id
    )
);

```

Learning Outcomes :

After completing this experiment, the following learning outcomes were achieved.

- **Correlated and Nested Subqueries:** Solved complex, multi-layered filtering problems (such as isolating the highest and second-highest departmental salaries) using correlated subqueries and nested SELECT statements.
- **Data Aggregation and Summarization:** Applied aggregate functions—specifically AVG(), MIN(), and MAX()—in conjunction with the GROUP BY clause to generate high-level analytical metrics.
- **Advanced Relational Joins:** Mastered the implementation of multi-table INNER JOIN operations to link related datasets (such as Departments, Students, and Marks) across primary and foreign key boundaries.